

Pemrograman Web Lanjut

SIB245007 | D-IV Sistem Informasi Bisnis

Pertemuan 10: Merancang dan Membangun REST API

Endpoint yang konsisten, autentikasi token, dan dokumentasi yang bisa dipercaya

Yang Akan Kamu Pelajari

1. Merancang endpoint REST API yang mengikuti semantik HTTP method secara konsisten, dan mengurasi bentuk respons lewat API Resource
2. Mengimplementasikan autentikasi berbasis token dengan Sanctum untuk klien yang tidak punya cookie jar, lengkap dengan cara klien itu sendiri mengonsumsi endpoint terproteksi
3. Mendokumentasikan kontrak API memakai spesifikasi OpenAPI, agar tim lain tidak perlu membaca kode controller untuk tahu cara memanggilnya

Deck ini membahas konsep. Menerapkan endpoint REST, Sanctum, dan OpenAPI pada Simple POS terjadi di jobsheet praktikum.

Bagian 1

Merancang Endpoint REST API

Istilah Penting (1/2)

REST (Representational State Transfer): gaya arsitektur API yang memetakan operasi ke resource lewat method HTTP standar (GET, POST, PUT, DELETE) dan URL yang merepresentasikan resource itu sendiri.

Endpoint: satu kombinasi method HTTP dan path URL yang menjalankan satu operasi spesifik, misalnya `POST /api/login` .

Token: string acak yang mewakili identitas pengguna setelah login berhasil, dikirim ulang pada setiap request berikutnya sebagai bukti bahwa pengguna itu sudah terautentikasi.

Istilah Penting (2/2)

Sanctum: paket autentikasi bawaan Laravel untuk API berbasis token, dirancang untuk klien SPA maupun klien mobile murni.

API Resource: kelas transformasi Laravel yang mengubah model Eloquent menjadi struktur JSON yang konsisten, memisahkan bentuk respons API dari struktur kolom tabel apa adanya.

REST bukan protokol baku dengan aturan ketat, melainkan sekumpulan konvensi yang disepakati bersama supaya API gampang ditebak perilakunya hanya dari method dan URL-nya.

Semantik HTTP Method Bukan Formalitas

GET harus aman dan idempoten

- Tidak mengubah state apa pun di server
- Dipanggil berkali-kali = efek sama seperti sekali
- Klien boleh mengulang request tanpa ragu

POST boleh mengubah state

- Bisa membuat baris baru di database
- Dipanggil dua kali bisa membuat dua baris
- Klien harus menangani risiko ini sendiri

GET `/api/products` yang terpanggil berkali-kali karena koneksi terputus tidak pernah menggandakan data. POST `/api/transactions` yang terpanggil dua kali karena request pertama timeout, memang berpotensi membuat dua transaksi, bukan sesuatu yang aman diasumsikan tidak terjadi.

API Resource: Mengurasi Bentuk Respons

Model Eloquent yang dikembalikan langsung sebagai JSON bisa membocorkan kolom internal (`updated_at` , `password`) atau relasi yang tidak relevan bagi klien. API Resource memastikan setiap endpoint mengembalikan bentuk JSON yang sudah dikurasi dan konsisten.

```
class ArticleResource extends JsonResource
{
    public function toArray($request): array
    {
        return [
            'id' => $this->id,
            'title' => $this->title,
            'author' => $this->author->name,
        ];
    }
}
```

Hanya tiga kolom yang dipilih untuk muncul di JSON, apa pun kolom tambahan yang kelak ditambahkan ke tabel `articles` tidak otomatis ikut bocor ke klien.

Bagian 2

Autentikasi Token dengan Sanctum

Sesi Cookie vs Token: Beda Klien, Beda Mekanisme

Halaman web (Pertemuan 7)

- Browser menyimpan cookie otomatis
- Sesi dicocokkan lewat cookie itu
- Cocok untuk klien yang punya cookie jar

Klien API (mobile, skrip luar)

- Tidak punya cookie jar seperti browser
- Token disimpan sendiri oleh klien
- Dikirim ulang eksplisit tiap request

Keduanya sama-sama membuktikan "siapa yang sedang request ini", hanya mekanisme penyimpanan dan pengiriman buktinya yang berbeda, menyesuaikan jenis klien yang memakainya.

Alur Autentikasi Sanctum



```
$user = Auth::user();  
$token = $user->createToken('klien-mobile')->plainTextToken;
```

`createToken()` menyimpan token itu dalam bentuk hash di tabel `personal_access_tokens`, dan `->plainTextToken` adalah satu-satunya kesempatan token mentah itu terlihat. Setelah respons login ini, server tidak bisa lagi menunjukkan ulang token yang sama dalam bentuk terbaca.

Melindungi Route dengan auth:sanctum

Route yang perlu dilindungi cukup dibungkus middleware `auth:sanctum`, mirip pola `role:admin` pada Pertemuan 7, hanya saja yang diperiksa kali ini adalah keberadaan dan keabsahan token lewat header `Authorization`, bukan sesi cookie.

```
Route::middleware('auth:sanctum')->group(function () {  
    Route::get('/articles', [ArticleController::class, 'index']);  
    Route::post('/articles', [ArticleController::class, 'store']);  
});
```

Tanpa header `Authorization: Bearer <token>` yang valid, middleware ini menolak request sebelum pernah sampai ke controller, persis seperti `role:admin` menolak pengguna yang bukan admin sebelum sampai ke aksi yang dilindungi.

Bagian 3

Mengonsumsi API dan Mendokumentasikannya

Klien Wajib Memeriksa Status Sebelum Membaca Isi

Klien API (aplikasi mobile, skrip PHP, atau bahasa apa pun) tidak pernah boleh mengasumsikan setiap request selalu berhasil.

```
$login = Http::post("{baseUrl}/api/login", [
    'email' => $email,
    'password' => $password,
]);

if ($login->failed()) {
    // tangani galat di sini, jangan lanjut ke langkah berikutnya
}

$token = $login->json('token');
```

Klien yang langsung membaca `$login->json('token')` tanpa memeriksa `failed()` lebih dulu gagal secara diam-diam: kredensial yang salah membuat `$token` bernilai `null` tanpa pernah jelas kenapa, dan galatnya baru meledak beberapa langkah kemudian di tempat yang membingungkan.

Membedakan Jenis Galat

Bukan semua galat ditangani dengan cara yang sama. Kode status HTTP memberi tahu klien tindakan apa yang tepat berikutnya.

```
match ($response->status()) {  
    201 => /* tersimpan, baca $response->json('data') */,  
    401 => /* token tidak valid, klien harus login ulang */,  
    422 => /* input ditolak, perbaiki input, bukan login ulang */,  
    default => /* galat lain, tampilkan ke pengguna */,  
};
```

401 berarti masalah ada pada identitas klien (token kedaluwarsa atau dicabut), sementara 422 berarti masalah ada pada data yang dikirim. Menyamaratakan keduanya sebagai "login ulang saja" membuat klien mengulang login tanpa guna saat masalahnya sebenarnya ada di input.

OpenAPI: Kontrak, Bukan Catatan yang Cepat Basi

OpenAPI: spesifikasi terstruktur (YAML atau JSON) yang mendeskripsikan endpoint API, bisa dibaca alat lain seperti generator dokumentasi interaktif atau generator kode klien.

```
paths:
  /api/articles:
    get:
      summary: Ambil daftar artikel
      responses:
        '200':
          description: Daftar artikel
```

Catatan teks biasa di README cepat basi karena tidak ada yang memaksa pembaruannya saat satu field respons berubah nama. Format terstruktur seperti OpenAPI bisa diperiksa alat otomatis, sehingga dokumentasi yang basi ketahuan lewat test yang gagal, bukan lewat keluhan tim lain belakangan.

Menerapkan pada Simple POS

Kamu akan merancang dan membangun endpoint `POST /api/login`, `GET /api/products`, dan `POST /api/transactions` untuk Simple POS di jobsheet praktikum: dilindungi token Sanctum, dikonsumsi lewat klien HTTP sungguhan, dan didokumentasikan dengan OpenAPI.

Kode lengkap: github.com/se-polinema/simple-pos-ch09

Rangkuman (1/2)

- Endpoint REST mengikuti semantik HTTP method secara konsisten: `GET` aman dan idempoten, `POST` boleh mengubah state dan berisiko tergandakan kalau diulang
- API Resource mengurasi bentuk JSON yang dikembalikan, memisahkan respons API dari struktur kolom tabel apa adanya
- Sanctum menerbitkan token lewat `createToken()`, dikirim klien lewat header `Authorization: Bearer`, diperiksa middleware `auth:sanctum` pada route yang dilindungi

Rangkuman (2/2)

- Klien API wajib memeriksa `failed()` sebelum membaca isi respons, dan membedakan penanganan 401 (login ulang) dari 422 (perbaiki input), bukan menyamaratakan semua galat
- Spesifikasi OpenAPI mendokumentasikan `paths`, `requestBody`, dan `responses` tiap endpoint dalam format terstruktur yang bisa diperiksa alat otomatis, bukan sekadar catatan pasif yang cepat basi

Referensi & Diskusi

Dokumentasi resmi Laravel Sanctum, spesifikasi OpenAPI, dan makalah Fielding tentang prinsip arsitektur REST

Kode lengkap: `github.com/se-polinema/simple-pos-ch09`

Pertemuan berikutnya: awal proyek PBL (Project Based Learning), kamu akan merencanakan fitur dan arsitektur proyek webmu sendiri